

A Configurable RISC-V Vector Processor with FSM-Driven Accelerator for Data-Intensive Workloads

Akkapolu Sankararao, Vashist Managari, Naveen Kollepara, Binsu J Kailath

Department of Electronics and Communication Engineering

IIITDM Kancheepuram, Chennai, Tamil Nadu - 600127, India

Email: ec23d0008@iiitdm.ac.in, ec21b1038@iiitdm.ac.in, ec21b1030@iiitdm.ac.in, bkailath@iiitdm.ac.in

Abstract—With the growing demand for scalable, parallel and energy-efficient computing in AI, machine learning (ML), high-performance embedded systems and security applications, RISC-V vector architectures are increasingly preferred over conventional scalar RISC-V cores. This paper presents a high-performance RISC-V vector architecture featuring a 512-bit register file, dynamic bit-width support and a pipelined matrix multiplication accelerator optimized for data-intensive workloads. A novel vector datapath—incorporating forward and reverse logic modules managed by an efficient finite state machine (FSM)—enables parallel vector-scalar execution and multi-precision capabilities within a compact hardware footprint. The proposed architecture integrates custom matrix multiplication instructions and has been implemented on a Xilinx Zynq-7000 FPGA, achieving up to 85.63% reduction in clock cycles across matrix sizes ranging from 2×2 to 32×32 . Additionally, benchmarking on high-bandwidth applications including convolution, FFT and linear regression demonstrates clock cycle reductions of 78.4%, 58.4% and 62.5%, respectively. These results outperform state-of-the-art RISC-V implementations, underscoring the design's scalability and suitability for high-throughput, real-time applications.

Index Terms—RISC-V, Vector ISA, Matrix multiplication, vector architecture, FSM, Memory Management, Accelerator, Custom Instructions

I. INTRODUCTION

As AI, ML and cybersecurity workloads demand greater computational efficiency, RISC-V has gained prominence for its extensibility and energy-efficient open-source design [1], [2]. Its modular ISA enables custom extensions and hardware-software co-design for high-performance use. The RISC-V Vector Extension (RVV) enhances parallelism, benefiting data-intensive tasks like matrix/vector operations in Edge-AI systems [3], [4].

Although vector processing substantially improves performance, it alone often remains insufficient for large-scale parallel workloads such as deep neural networks (DNNs), cryptographic operations and high-bandwidth streaming. To address this, specialized hardware accelerators are integrated to offload compute-intensive vector operations, enhancing data-level parallelism and enabling faster matrix multiplications, hashing and encryption with reduced latency, higher throughput and lower power consumption [5]–[8]. However, simply combining vector ISAs with accelerators can lead to

latency and memory overhead from issues such as contention and suboptimal cache utilization. Solutions such as RISC-V² [5] and variable bit-precision extensions [6] have partially addressed these challenges through parallel memory pipelines and configurable precision, yet problems like idle cores and limited scalar-vector concurrency persist [9] [10] [11] [12].

To overcome these limitations, this paper proposes a novel optimized RISC-V vector architecture that integrates an interrupt-driven FSM and advanced memory management features. A pipelined matrix multiplication accelerator is developed to demonstrate the architecture's capabilities. As shown in Fig. 1, the system features dedicated control logic for concurrent scalar-vector execution, forward and reverse data transfer mechanisms and direct D-cache access. This design enables high-throughput operations with significantly reduced clock cycles, achieving up to 62.5% performance gains over conventional RISC-V architectures while enhancing the efficiency in both ML and cryptographic workloads.

The rest of the paper is organized as follows: Section II presents the architectural design and hardware modules, Section III discusses performance metrics and comparisons and Section IV concludes the work and outlines future research directions.

II. PROPOSED RISC-V WITH VECTOR ARCHITECTURE

This work proposes an extended RISC-V architecture that combines the base RISC-V ISA with a vector extension. The primary features are as follows:

- 1) A dedicated memory management unit (MMU) for efficient data transfers between the accelerator, vector register file (VREG) and Data(D)-cache.
- 2) Support for concurrent scalar execution alongside vector operations.
- 3) Implementation of front and reverse logic modules with FSM control for parallel data loading and storing

These features have been implemented and integrated into the RISC-V architecture, as illustrated in Fig. 1. Key highlights include support for direct data loading from the D-Cache by both the vector architecture and RISC-V core, dedicated vector-specific ALU modules extended up to 64 bits and built-in support for fixed-point and floating-point conversion. The

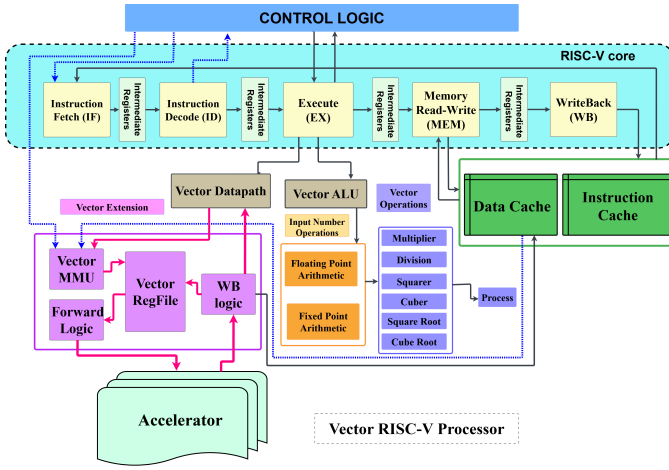


Fig. 1. Overview of the proposed RISC-V vector processor architecture with integrated accelerator and FSM control

design also incorporates a pipelined architecture to enhance performance.

The novelty of the vector architecture and the accelerator unit has been explained below.

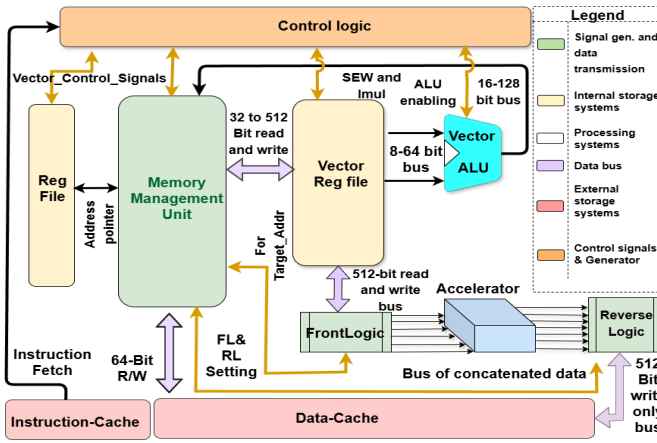


Fig. 2. Block diagram of the RISC-V vector datapath including internal buses, control, and FSM-based data handling

A. Direct D-Cache memory access through MMU

Traditional data transfer mechanisms between the D-cache and RISC-V processor often suffer from inefficiencies and high latency. Prior works [15] have proposed dedicated MMUs for direct data transfer to accelerators; however, such designs frequently result in processor stalling, as computation must wait for the entire dataset to load. The proposed architecture mitigates these limitations through optimizations that improve data throughput and CPU utilization. The system includes 32 vector registers, each 512 bits wide, supporting operand sizes from 8 to 512 bits. The MMU manages high-bandwidth data flow among the vector registers, ALU and accelerator modules using immediate addressing, enabling low-latency transfers without processor interruption. Additionally, a 64-bit read/write signal facilitates direct D-cache access. For

example, when transferring 16-bit data, the MMU configures the bus to store four 16-bit data sets within the MMU module. This identification of data width is performed by the associated vector instruction, enabling continuous streaming as shown in Fig. 2.

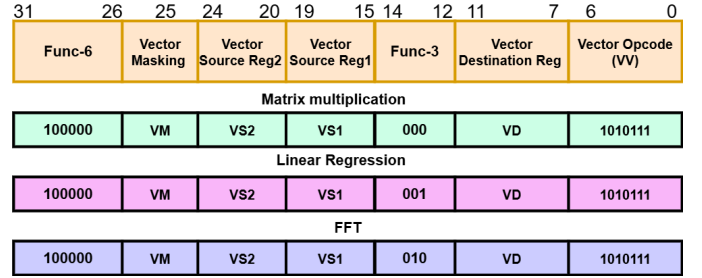


Fig. 3. Custom 32-bit instruction formats for vector-matrix operations and concurrent scalar execution

B. Concurrent scalar operations with vector instruction

To prevent processor idling during accelerator usage, the architecture allows concurrent scalar execution alongside vector operations. The MMU dynamically configures the vector register file to load and store configurable numbers of 32-bit or 64-bit vectors, accommodating up to 512 bits of input data per cycle. The key features are as follows:

- 1) Stall the program counter(PC) when a vector instruction is identified
- 2) Identify opcodes for respective operation for initiation of accelerator and ALU unit
- 3) Implement load operations from D-cache to VREG.
- 4) Executes the parallel modules from parallel VREG's using Front Logic module
- 5) Compute and store the results into the VREG from the accelerator
- 6) Once the accelerators 1st computation is done for a given set of data, the respective Reverse Logic will store the output back to D-cache
- 7) Resume the normal pipeline for the next set of instructions of RISC-V.

In order to permit parallel execution of vector and scalar operations, the specific instruction formats has been devised for each of the three customized vector instruction in Fig. 3. An example of this is given as Algorithm 1 which demonstrates concurrent calling of vector instruction with its extension along with its scalar instruction.

Algorithm Assembly-Level Implementation of Concurrent Scalar and Vector Instructions

- 0: **Input:** Vector matrix multiplication along with an add operation and a subtraction operation concatenated
- 0: `vmatmul.v vd_1, vs_1, vs_2`
- 0: `vmatmul.v vaddw.v vd_2, vs_3, vs_4`
- 0: `vmatmul.v vsubw.v vd_2, vs_3, vs_4`
- 0: **Output:** Parallel computation of the 3 instructions without stalling the processor =0

C. Front-logic, Reverse-logic and FSM implementation for configuring the accelerator

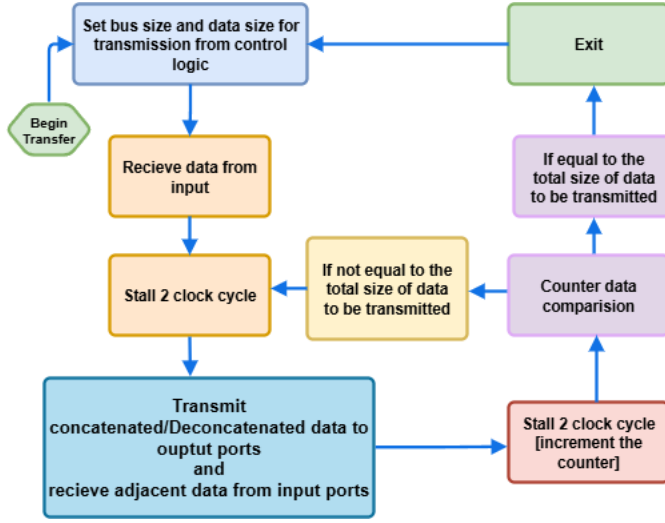


Fig. 4. FSM managing front and reverse logic for efficient data transfer between the MMU and D-cache

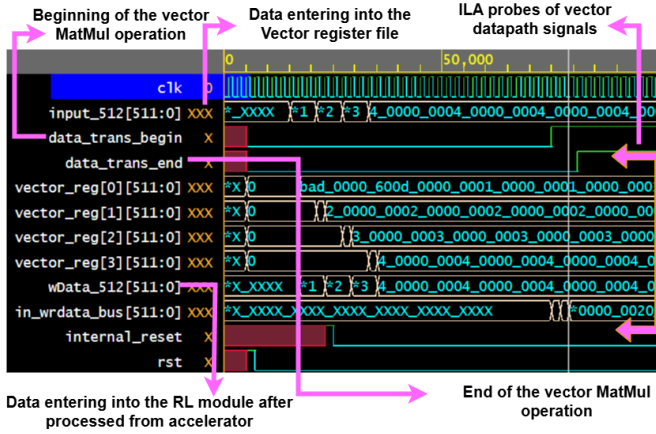


Fig. 5. Timing waveform of a 4x4 matrix multiplication operation executed in 28 clock cycles.

Conventional approaches [5], [15], store the entire dataset in a register file before serial transfer to the accelerator, resulting in CPU idling. To overcome this, a dedicated data loading and storing module enables direct transfers without CPU intervention. A finite state machine (FSM), manages the front and reverse logic modules according to the data format (e.g., IEEE-754, fixed-point, concatenated up to 512 bits). The FSM remains active during processing and transitions to idle upon completion. This pipelined design overlaps FSM control and memory operations to improve throughput. The MMU sets FSM parameters, which coordinates stalling, data routing, format adaptation and bit-width scaling (8–512 bits). Fig. 4 illustrates the timing and control flow.

The MMU sets FSM parameters which controls stalling, directing data to the shared registers in the D-cache or VREG.

The logic modules support data filtering, format adaptation, concatenation and bit-width scaling (8 to 512 bits), all configured via the MMU. Fig. 5 illustrates this, where 512-bit buses perform load/store operations and MatMul as the ‘wData_512’ signal stalls and the ‘in_wrdata_bus’ signal initiates computation. During the operation of FSM, parallel sets of data is being transmitted between each vector_reg stalls. Once the data is processed, the data is then stalled by the data_trans_end signals and the final set of data is sent back to the D-Cache Memory with the ‘in_wrdata_bus’ bus.

D. Custom Instruction Design for Matrix Multiplication Accelerator

To evaluate the efficiency of the vector architecture in handling data load and store operations, matrix multiplication (MatMul) with variable bit-width is selected. MatMul is both compute-intensive and memory-bound, requiring frequent access to rows and columns of input matrices for computing each output element. The integrated architecture of the vector processor and matrix multiplier is shown in Fig. 6. The MatMul operation is controlled by a sequential logic flow, driven by vector instruction bits sent to multiple modules at once. These bits manage key operations such as bus width selection, vector register control and enabling or disabling of the ALU and accelerator.

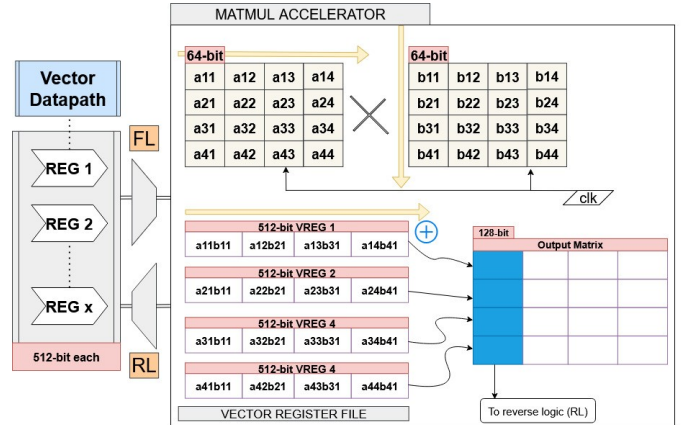


Fig. 6. Architecture of the 4x4 matrix multiplication (64-bit each) accelerator integrated with the vector register file and control logic.

An interrupt controller is also included to handle the step-by-step transfer of processed data to the output register and D-cache after execution. The detailed steps of this process are shown in Fig. 7.

III. EVALUATION RESULTS

The proposed RISC-V vector processor and its integrated accelerator modules were implemented and validated on the Xilinx Zynq-7000 AP SoC XC7Z020-CLG484 FPGA using Vivado and Vitis 2020.2 toolchains. The designs have been implemented and verified at 100 MHz of clock frequency. The hardware synthesis results are summarized in Table I.

Conventional Vector processors in [9], [13], [14] are widely used for data-intensive algorithms due to their ability to

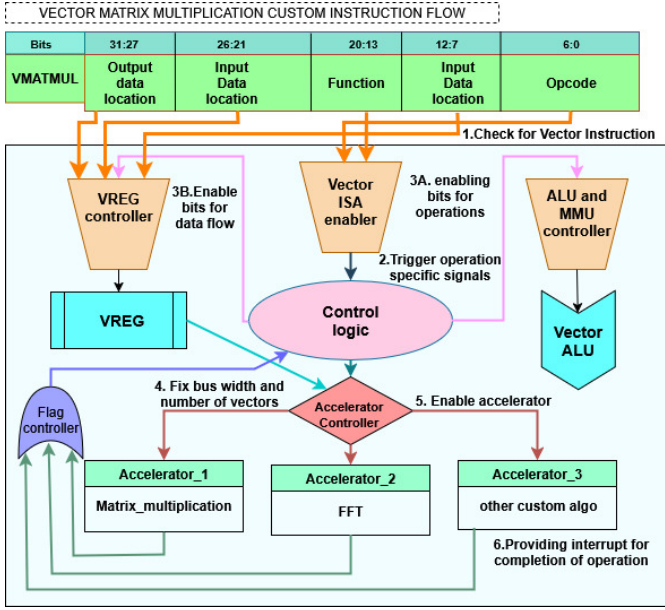


Fig. 7. Control logic coordination with vector architecture for implementing custom vector instructions

TABLE I
FPGA RESOURCE UTILIZATION OF THE PROPOSED RISC-V VECTOR ARCHITECTURE

FPGA Parameter	Used	Available	Utilization
LUT	11747	53200	22.08%
FF	3	220	1.13%
BRAM	18	140	12.86%

improve performance and efficiency. This has led to advancements in vector architectures that reduce bandwidth bottlenecks, area and power consumption while improving instruction throughput. In this paper, these metrics are used to evaluate the proposed vector architecture with integrated accelerators.

TABLE II
COMPARATIVE ANALYSIS OF CLOCK CYCLES FOR HIGH BANDWIDTH OPERATIONS

Algorithms and architectures with number of clock cycles	Convolution	FFT	Linear regression
RV ² 4 lane [5]	272	78	1130
Proposed RISC-V with vector	315	125	2240
Proposed RISC-V with accelerator and vector	68	52	840

To evaluate computational efficiency, high-bandwidth operations such as convolution, Fast Fourier Transform (FFT) and linear regression were implemented and benchmarked across various state-of-the-art architecture: standard scalar RISC-V [1], RISC-V² vector architecture [5], proposed RISC-V with vector extension and the proposed RISC-V with vector extension and accelerator integration. As detailed in Table II, the proposed vector architecture with accelerator achieves a 78.4% reduction in clock cycles for convolution, 58.4% for FFT and

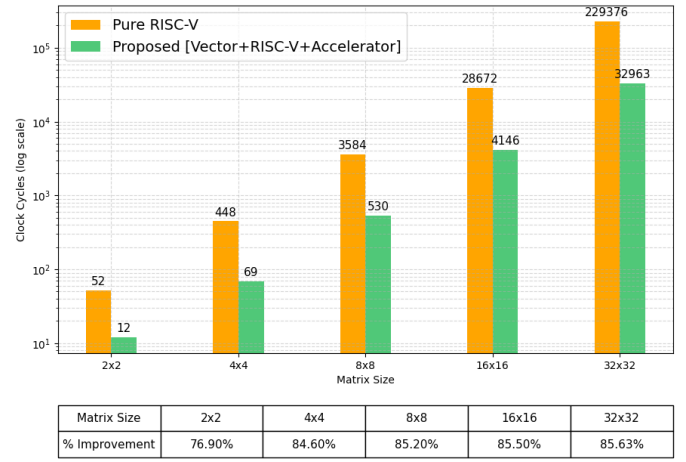


Fig. 8. Clock cycles comparison for matrix multiplication across different vector accelerator configurations.

TABLE III
QUALITATIVE COMPARISON WITH OTHER APPROXIMATE VECTOR ARCHITECTURES

	This work	Ref. [5]	Ref. [6]
Implementation or Process	FPGA	45nm	FPGA
ISA	RV32I - Out-of-order	RV32I - out-of-order	RV32I - In-order
Number of stages	5 stage	5 Stage	5 stage
Target for architectural Innovation	Pipelining accelerator and superscalar optimization	Register remapping and lane architecture	variable bit precision with vector accelerator
Approximation methods	MMU Innovation, custom ISA FSM.	Dedicated dataflow logics.	Processing element integration.

62.5% for linear regression, compared to the baseline vector implementation. These results demonstrate the effectiveness of the custom instructions and pipelined Datapath in reducing latency for real-time workloads. [5], [6] Matrix multiplication accelerators sized 2x2 to 32x32 were implemented with custom vector instructions, substantially reducing clock cycles versus standard RISC-V multipliers (Fig. 8). Performance gains of 76.9% to 85.6% across sizes confirm scalability, ideal for AI/ML inference and high-throughput tasks. A qualitative comparison with other approximate vector architectures is presented in Table III. Compared to existing designs in [5] and [6], the proposed architecture introduces a unique combination of pipelined accelerators, MMU integration and dual-direction vector datapath control via FSM. These innovations, along with custom instruction set extensions, enable efficient matrix processing, reduced instruction overhead and better adaptability to high-bandwidth workloads, while maintaining compliance with RISC-V standards.

IV. CONCLUSION AND FUTURE SCOPE

This paper presented an optimized RISC-V vector processor architecture integrated with accelerators for high-throughput applications. Implemented on Xilinx Zynq-7000 FPGA operating at 100 MHz clock frequency, the design achieved up to

85.63% reduction in clock cycles for matrix operations and substantial performance improvements in convolution, FFT and linear regression. Future work will explore extending the instruction set to support complex domain-specific workloads in DSP, cryptography, and real-time embedded systems, further enhancing performance and flexibility while maintaining RISC-V compliance.

REFERENCES

- [1] A. Raveendran, V. B. Patil, D. Selvakumar and V. Desalpine, "A RISC-V instruction set processor-micro-architecture design and analysis," 2016 International Conference on VLSI Systems, Architectures, Technology and Applications (VLSI-SATA), Bengaluru, India, 2016, pp. 1-7, doi: 10.1109/VLSI-SATA.2016.7593047.
- [2] D. R. Ditzel and the Esperanto team, "Accelerating ML Recommendation With Over 1,000 RISC-V/Tensor Processors on Esperanto's ET-SoC-1 Chip," in *IEEE Micro*, vol. 42, no. 3, pp. 31-38, 1 May-June 2022, doi: 10.1109/MM.2022.3140674.
- [3] M. Johns and T. J. Kazmierski, "A Minimal RISC-V Vector Processor for Embedded Systems," 2020 Forum for Specification and Design Languages (FDL), Kiel, Germany, 2020, pp. 1-4, doi: 10.1109/FDL50818.2020.9232940.
- [4] S. Jiang, Y. Zou, H. Wang and W. Li, "An FFT Accelerator Using Deeply-coupled RISC-V Instruction Set Extension for Arbitrary Number of Points," 2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors (ASAP), Porto, Portugal, 2023, pp. 165-171, doi: 10.1109/ASAP57973.2023.00036.
- [5] K. Patsidis, C. Nicopoulos, G. C. Sirakoulis and G. Dimitrakopoulos, "RISC-V2: A Scalable RISC-V Vector Processor," 2020 IEEE International Symposium on Circuits and Systems (ISCAS), Seville, Spain, 2020, pp. 1-5, doi: 10.1109/ISCAS45731.2020.9181071.
- [6] R. RK, S. Sinha and N. Rao, "Variable Bit-Precision Vector Extension for RISC-V Based Processors," 2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MC-SoC), Singapore, Singapore, 2021, pp. 114-121, doi: 10.1109/MC-SoC51149.2021.00024.
- [7] C. Wang, C. Fang, X. Wu, Z. Wang and J. Lin, "SPEED: A Scalable RISC-V Vector Processor Enabling Efficient Multiprecision DNN Inference," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 1, pp. 207-220, Jan. 2025, doi: 10.1109/TVLSI.2024.3466224.
- [8] M. Perotti, M. Cavalcante, N. Wistoff, R. Andri, L. Cavigelli and L. Benini, "A "New Ara" for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design," 2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP), Gothenburg, Sweden, 2022, pp. 43-51, doi: 10.1109/ASAP54787.2022.00017.
- [9] M. Ali, M. von Ameln and D. Goehring, "Vector Processing Unit: A RISC-V based SIMD Co-processor for Embedded Processing," 2021 24th Euromicro Conference on Digital System Design (DSD), Palermo, Italy, 2021, pp. 30-34, doi: 10.1109/DSD53832.2021.00014.
- [10] A. Fernandes, L. Crespo, N. Neves, P. Tomás, N. Roma and G. Falcao, "Functional Validation of the RISC-V Unlimited Vector Extension," in *IEEE Embedded Systems Letters*, vol. 17, no. 1, pp. 2-5, Feb. 2025, doi: 10.1109/LES.2024.3416820.
- [11] V. Razilov, E. Matuš and G. Fettweis, "Communications Signal Processing Using RISC-V Vector Extension," 2022 International Wireless Communications and Mobile Computing (IWCMC), Dubrovnik, Croatia, 2022, pp. 690-695, doi: 10.1109/IWCMC55113.2022.9824961.
- [12] V. Maisto and A. Cilaro, "A Pluggable Vector Unit for RISC-V Vector Extension," 2022 Design, Automation Test in Europe Conference Exhibition (DATE), Antwerp, Belgium, 2022, pp. 1143-1148, doi: 10.23919/DATE54114.2022.9774501.
- [13] N. Kovačević, . Mišeljčić and A. Stojković, "RISC-V vector processor for acceleration of machine learning algorithms," 2022 30th Telecommunications Forum (TELFOR), Belgrade, Serbia, 2022, pp. 1-4, doi: 10.1109/TELFOR56187.2022.9983779.
- [14] C. Imianosky, D. A. Santos, D. R. Melo, F. Viel and L. Dilillo, "Special Session: Reliability and Performance Evaluation of a RISC-V Vector Extension Unit for Vector Multiplication," 2024 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), Didcot, United Kingdom, 2024, pp. 1-6, doi: 10.1109/DFT63277.2024.10753524.
- [15] V. N. Chander and K. Varghese, "A Soft RISC-V Vector Processor for Edge-AI," 2022 35th International Conference on VLSI Design and 2022 21st International Conference on Embedded Systems (VLSID), Bangalore, India, 2022, pp. 263-268, doi: 10.1109/VLSID2022.2022.00058.